

# Microkernel in Operating System

A Minimalistic kernel with essential functions

## Struktur Sistem Operasi



Mata Kuliah Sistem Operasi

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom

Program Studi Teknik Informatika

Universitas Muhammadiyah Jember

2025

# ⚙️ Operating-System Services

Layanan sistem operasi adalah **fasilitas dasar** untuk memudahkan pengguna dan program



## Eksekusi Program

Memuat, menjalankan, dan menghentikan proses



## Operasi I/O

Akses ke perangkat keras: keyboard, mouse, monitor, printer



## Manipulasi File

Membaca, menulis, membuat, dan menghapus file



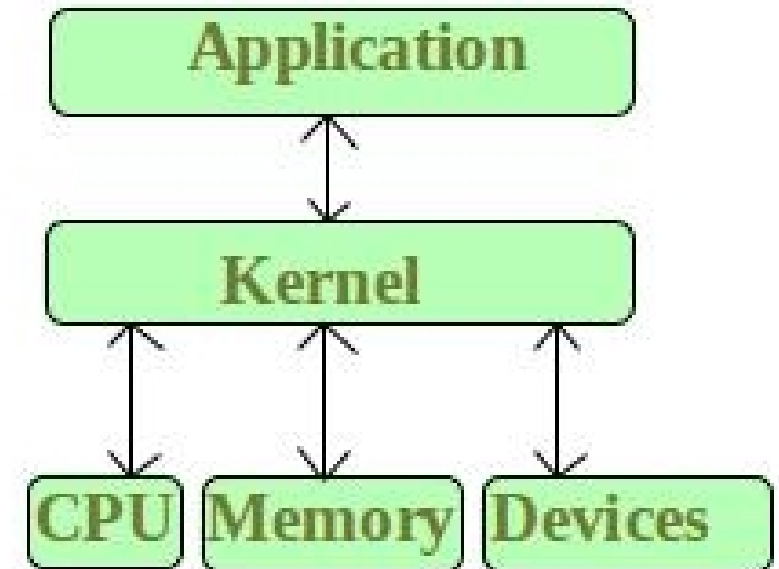
## Komunikasi

Pertukaran informasi antar proses atau komputer



## Deteksi Kesalahan

Mendeteksi dan menangani kesalahan hardware dan software



# User and Operating-System Interface

Antarmuka sistem operasi adalah **jembatan komunikasi** antara pengguna dan sistem operasi



## Command Line Interface (CLI)

Antarmuka berbasis teks dimana pengguna mengetik perintah untuk mengontrol sistem

Windows CMD

Linux Terminal

PowerShell



## Graphical User Interface (GUI)

Antarmuka visual dengan ikon, jendela, dan menu yang dapat diakses dengan mouse atau sentuhan

Windows

macOS

Ubuntu Desktop



## Touch-based Interface

Antarmuka yang dirancang khusus untuk perangkat dengan layar sentuh

Android

iOS

Windows Touch



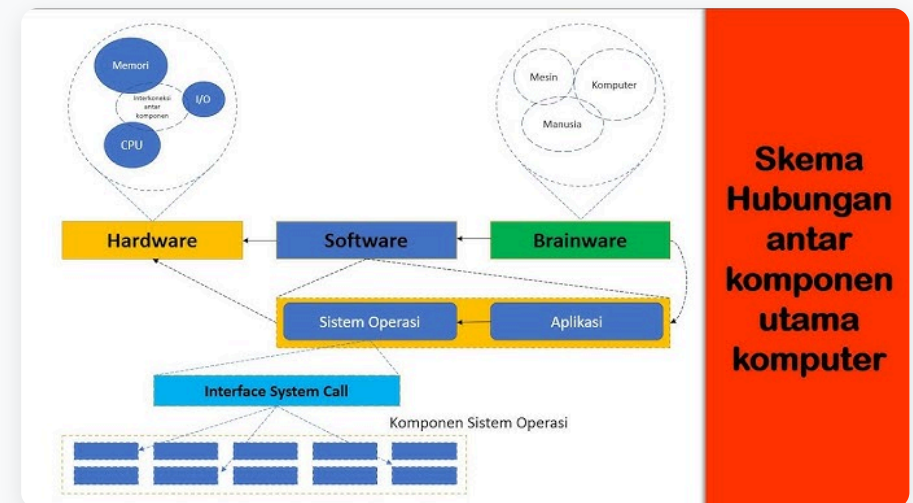
## Programmatic Interface (API)

Antarmuka yang memungkinkan aplikasi berkomunikasi dengan sistem operasi melalui pemanggilan fungsi

Win32 API

POSIX

System Calls



# 📞 System Calls

System Calls adalah **antarmuka pemrograman** yang memungkinkan program aplikasi meminta layanan dari kernel sistem operasi



## Proses Kontrol

Membuat, mengakhiri, dan mengelola proses



## Manipulasi File

Membaca, menulis, membuat, dan menghapus file



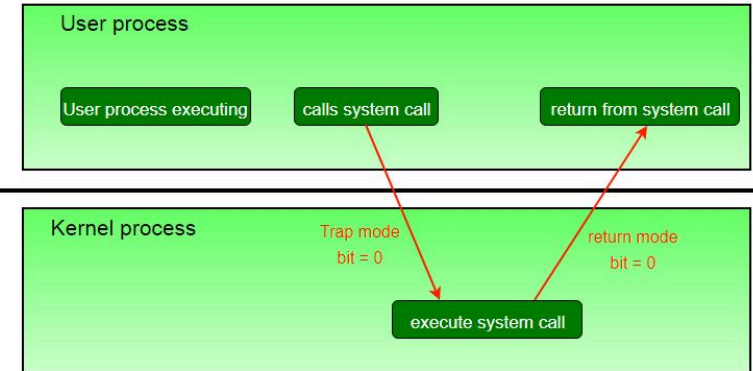
## Manajemen Perangkat

Mengakses dan mengontrol perangkat keras



## Komunikasi

Menghubungkan proses atau komputer dalam jaringan



## <> Contoh System Calls

fork()

exec()

open()

read()

write()

close()

pipe()

socket()

# System Services

System Services adalah **layanan latar belakang** yang menyediakan fungsionalitas dasar untuk sistem operasi dan aplikasi



## Layanan Keamanan

Autentikasi, otorisasi, dan enkripsi



## Layanan Jaringan

Protokol komunikasi dan konektivitas



## Layanan Penjadwalan

Manajemen tugas dan proses otomatis



## Layanan Penyimpanan

Manajemen disk dan backup data



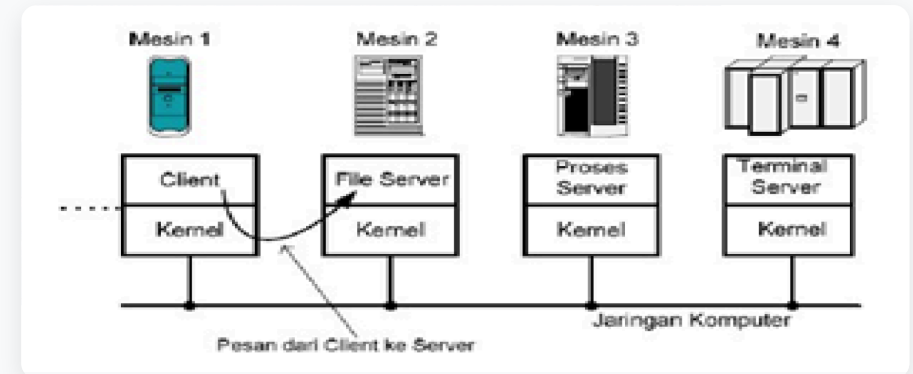
## Layanan Pencetakan

Manajemen antrian dan driver printer



## Layanan Pembaruan

Pembaruan sistem dan patch keamanan



## System Calls vs System Services

### System Calls

Antarmuka langsung ke kernel, digunakan oleh aplikasi

### System Services

Proses latar belakang, berjalan independen

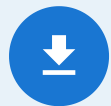
# Linkers and Loaders

Linkers dan Loaders adalah **program utilitas** yang memungkinkan program komputer dijalankan dengan benar



## Linker

Menggabungkan **beberapa file objek** menjadi satu program executable



## Loader

Memuat program dari **disk ke memori** dan mempersiapkannya untuk eksekusi



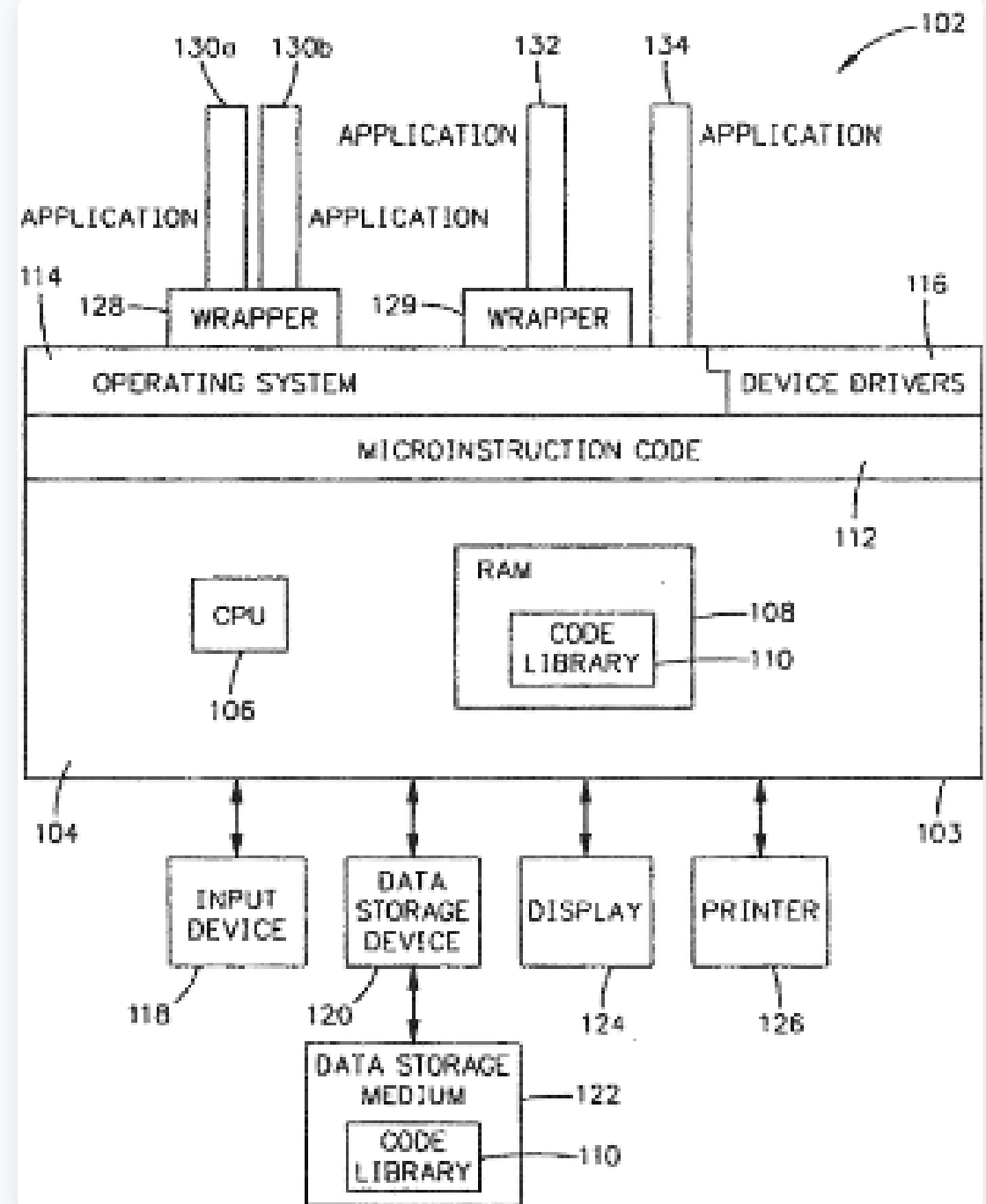
## Dynamic Linking

Menghubungkan program dengan **library saat runtime**, bukan saat kompilasi

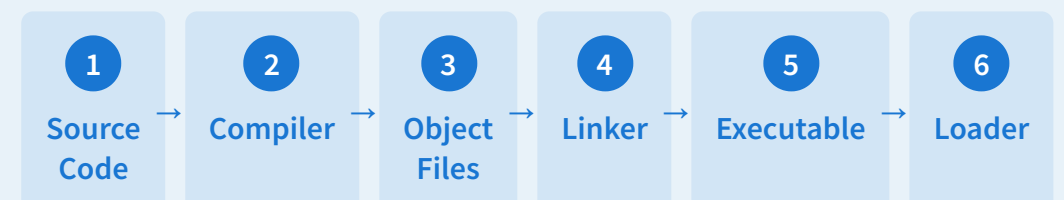


## Relocation

Menyesuaikan **alamat memori** program saat dimuat ke lokasi yang berbeda



## Proses Kompilasi dan Eksekusi



# Why Applications Are OS Specific

Aplikasi seringkali **tidak dapat berjalan** di sistem operasi yang berbeda karena alasan fundamental



## System Calls Berbeda

Setiap OS memiliki **antarmuka pemrograman** unik untuk mengakses layanan sistem



## Manajemen Memori

Cara OS mengalokasikan dan mengelola **ruang memori** berbeda antar sistem



## Driver Perangkat

Setiap OS memerlukan **driver khusus** untuk berkomunikasi dengan hardware



## Format File Eksekusi

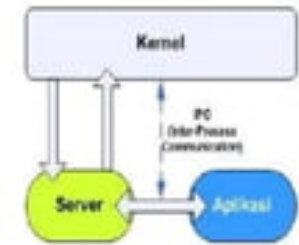
Windows (.exe), Linux (ELF), dan macOS (Mach-O) memiliki format berbeda

## SKEMA KERNEL

Monolithic Kernel



Microkernel



## Solusi Kompatibilitas



Emulator



Virtual Machine



Cross-Platform



Compatibility Layer

# Operating-System Design and Implementation

Desain dan implementasi sistem operasi adalah proses **kompleks** yang memerlukan perencanaan matang dan pertimbangan berbagai faktor



## Performa

Kecepatan respons dan efisiensi penggunaan **sumber daya** sistem



## Keamanan

Perlindungan data dan **kontrol akses** terhadap sumber daya sistem



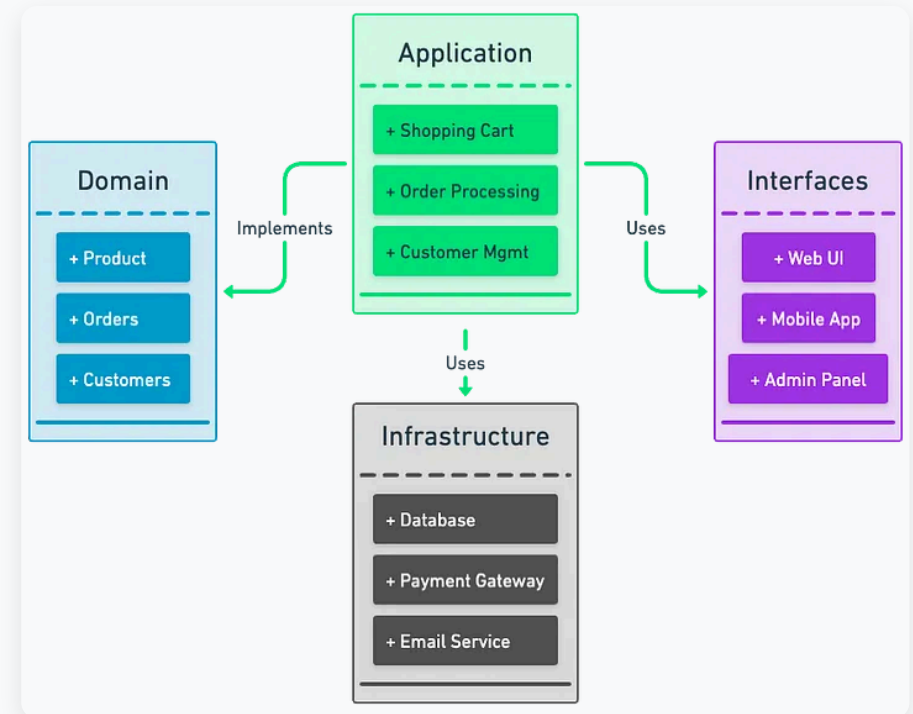
## Ekstensibilitas

Kemampuan sistem untuk **beradaptasi** dengan teknologi baru



## Portabilitas

Kemudahan memindahkan sistem ke **platform hardware** yang berbeda



## Tahap Implementasi

- 1 Definisi tujuan dan kebutuhan
- 2 Pemilihan arsitektur sistem
- 3 Desain antarmuka dan API
- 4 Implementasi komponen inti
- 5 Pengujian dan debugging



# Operating-System Structure

Struktur sistem operasi menentukan bagaimana **komponen** diorganisasikan dan berinteraksi



## Struktur Monolitik

Semua fungsi dalam **satu kernel** tanpa pemisahan yang jelas



## Struktur Berlapis

Dibagi menjadi **lapisan-lapisan** hierarkis dengan antarmuka jelas



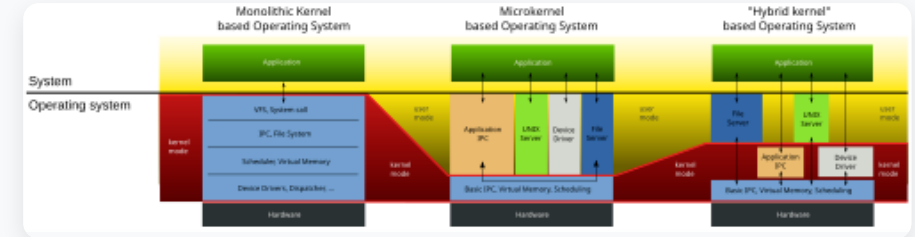
## Struktur Mikrokernel

Kernel **minimalis** dengan layanan lain sebagai proses terpisah



## Struktur Client-Server

Pemisahan fungsi menjadi **klien** dan **server** dengan peran berbeda



## Faktor Pemilihan Struktur



Performa



Keamanan



Kemudahan Pengembangan



Target Platform

# 🔧 Building and Booting an Operating System

Proses **membangun dan mem-boot** sistem operasi melibatkan beberapa tahapan penting untuk memastikan sistem berjalan dengan benar



## Kompilasi Kernel

Mengubah **source code** kernel menjadi format executable dengan konfigurasi yang tepat



## Bootloader

Program kecil yang **memuat kernel** ke memori dan memulai eksekusinya saat komputer dinyalakan



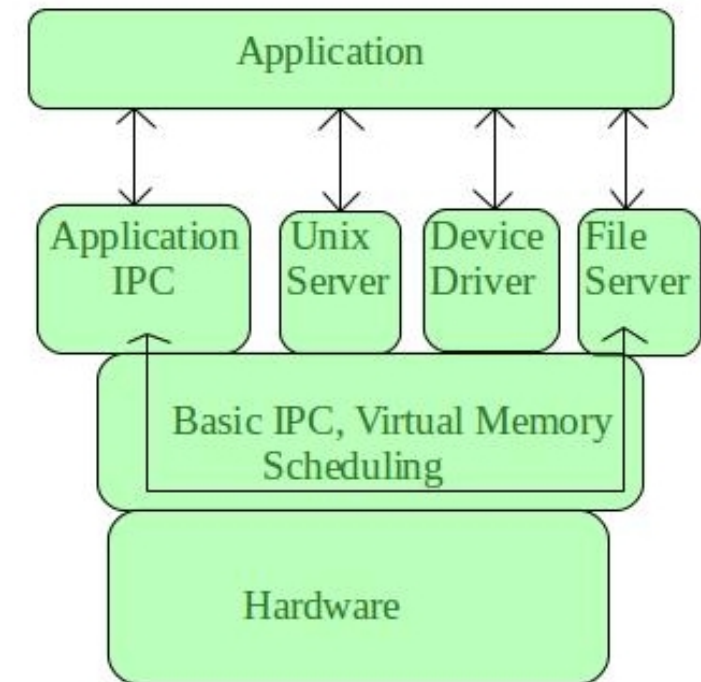
## Inisialisasi Sistem

Kernel menginisialisasi **hardware**, mengatur struktur data, dan memulai proses pertama



## Start User Space

Memulai **proses sistem** dan layanan yang diperlukan untuk menjalankan aplikasi pengguna



## ► Urutan Proses Boot

- 1 BIOS/UEFI POST
- 2 Bootloader Loading
- 3 Kernel Initialization
- 4 Device Driver Loading
- 5 System Services Start
- 6 User Interface Ready

# 🐛 Operating-System Debugging

Debugging sistem operasi adalah proses **mengidentifikasi dan memperbaiki kesalahan** dalam kode sistem operasi



## Source-Level Debugging

Menggunakan **debugger** untuk melacak eksekusi kode baris per baris



## Kernel Debugging

Menganalisis **memori dan register** untuk menemukan kesalahan di level kernel



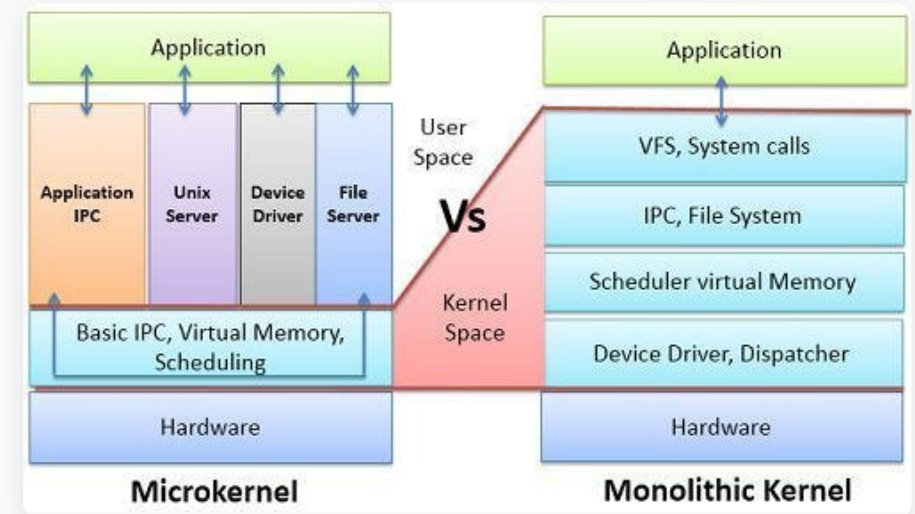
## Performance Analysis

Mengidentifikasi **bottleneck** dan masalah performa dalam sistem



## Log Analysis

Memeriksa **catatan sistem** untuk menemukan pola kesalahan



## Alat Debugging Umum



GDB



WinDbg



DTrace



Syslog

# Arsitektur Sistem Operasi

Arsitektur sistem operasi adalah **struktur organisasi** yang menentukan bagaimana komponen-komponen sistem operasi diatur dan berinteraksi



## Monolitik

Semua layanan dalam **satu kernel**



## Berlapis

Struktur **hierarkis** dengan lapisan-lapisan



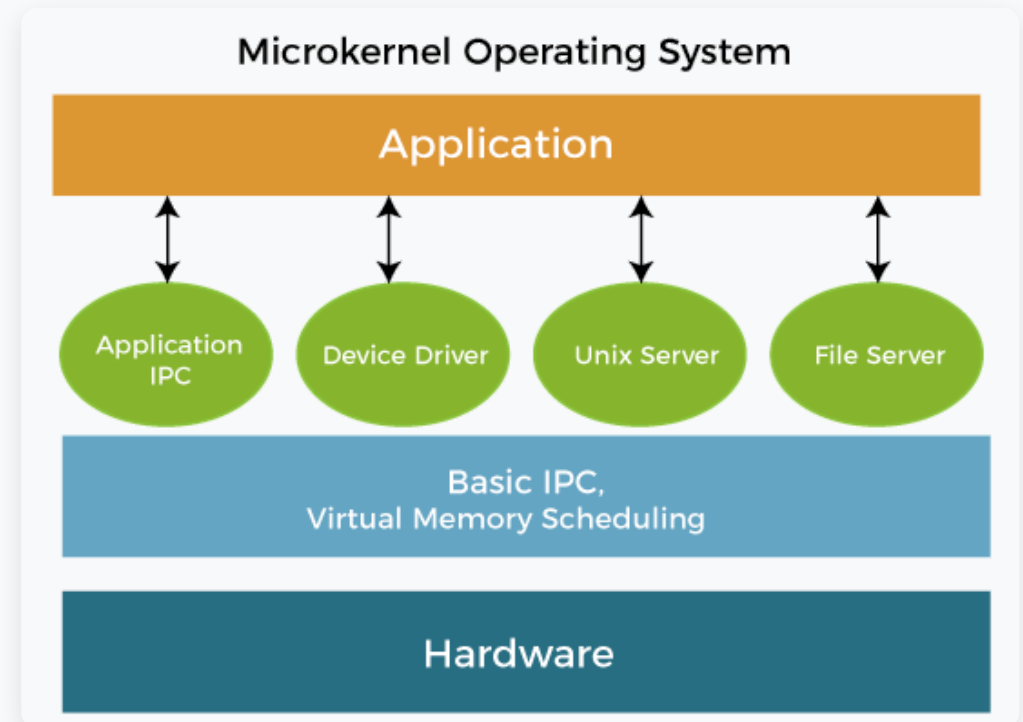
## Mikrokernel

Kernel **minimalis** dengan layanan terpisah



## Client-Server

Pemisahan antara **klien** dan **server**



# Arsitektur Monolitik

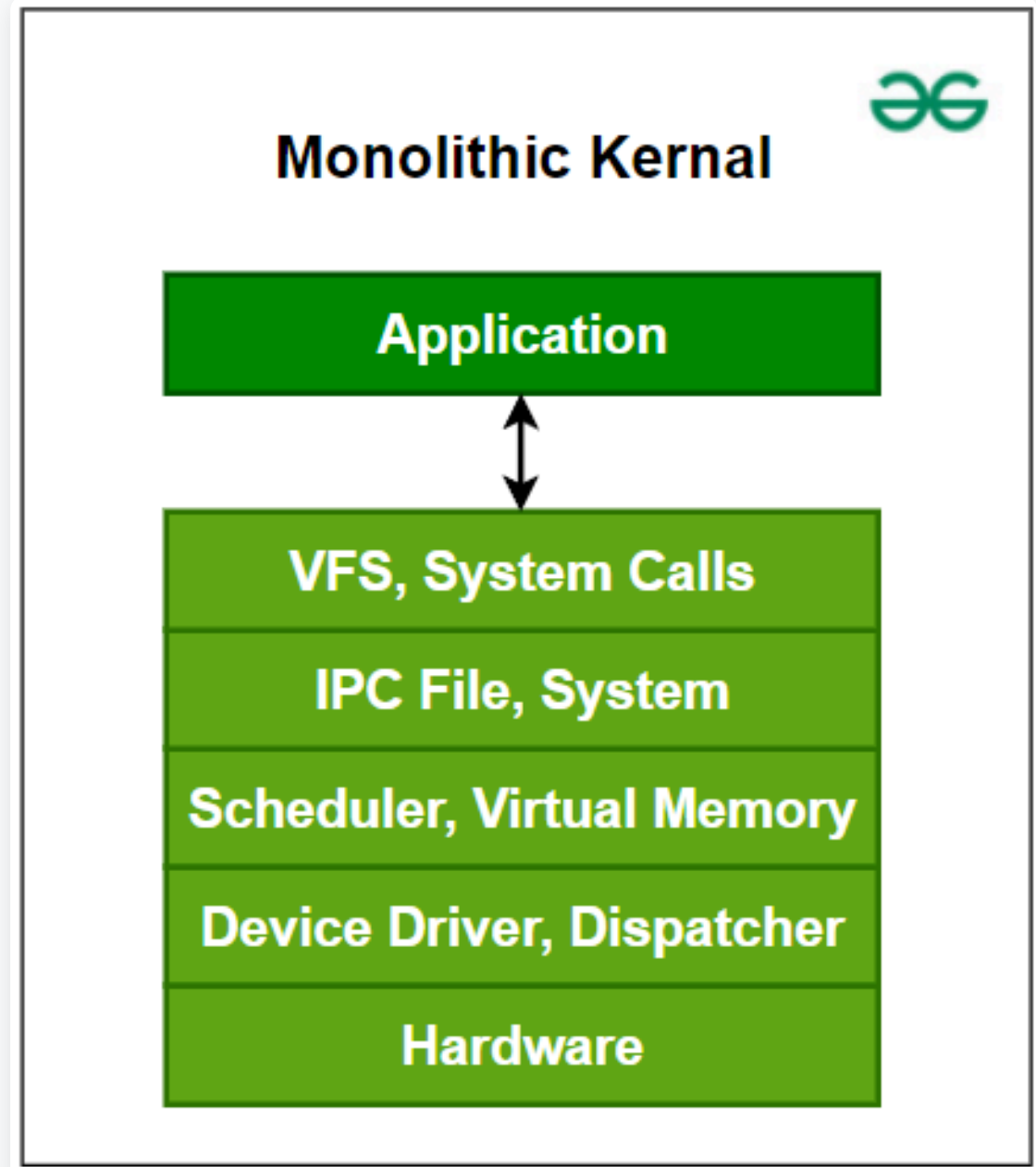
Sistem operasi sebagai **kumpulan prosedur** yang dapat saling dipanggil, dengan semua layanan berada dalam **satu kernel**

## Kelebihan

- Eksekusi **cepat** karena semua dalam satu ruang alamat
- Komunikasi antar komponen **efisien**
- Implementasi **sederhana**

## Kekurangan

- Sulit **debugging** dan modifikasi
- Keamanan **lebih rendah**
- Satu kesalahan dapat **menghentikan sistem**



## Contoh Sistem Operasi

Unix, Linux, MS-DOS, dan sistem operasi awal lainnya

# Arsitektur Berlapis

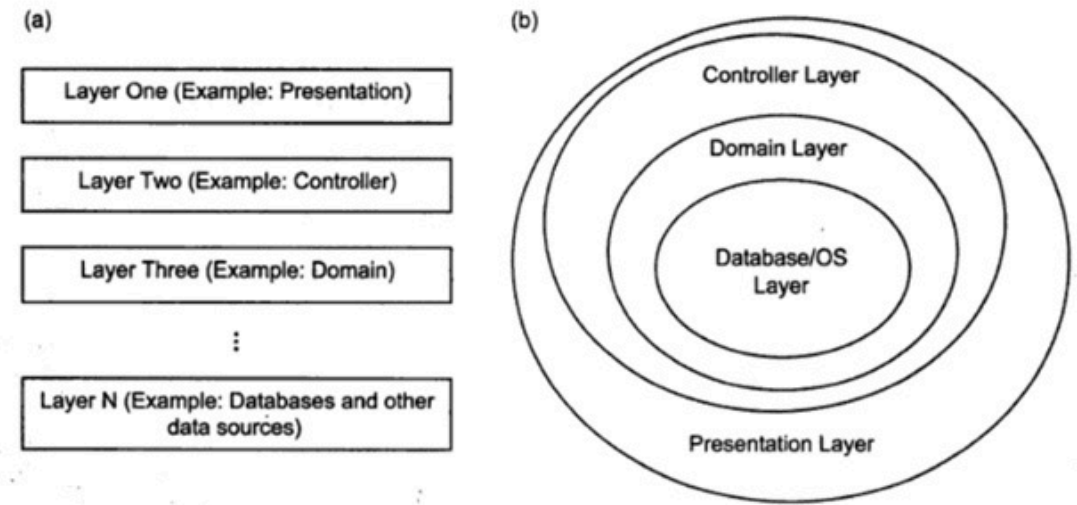
Sistem operasi yang dibentuk secara **hierarkis** berdasarkan lapisan-lapisan, dimana lapisan bawah memberikan layanan kepada lapisan di atasnya

## Kelebihan

- Desain **modular** yang terstruktur
- Setiap lapisan dapat **dikembangkan secara independen**
- Keamanan **lebih baik** dengan isolasi lapisan

## Kekurangan

- Performa **lebih lambat** karena komunikasi antar lapisan
- Perencanaan **lebih kompleks**
- Kurang **fleksibel** untuk perubahan fungsi



## Contoh Sistem Operasi

THE (Technische Hogeschool Eindhoven) OS

5

**Operator** Pengguna sistem

4

**User Programs** Aplikasi pengguna

3

**I/O Management** Mengelola perangkat I/O

2

**Operator Communication** Komunikasi antar proses

1

**Memory Management** Alokasi memori

0

**Processor Allocation** Penjadwalan CPU

# Arsitektur Mikrokernel

Kernel **minimalis** yang hanya berisi fungsi-fungsi dasar, dengan layanan lainnya berjalan sebagai **proses terpisah** di ruang pengguna

## Kelebihan

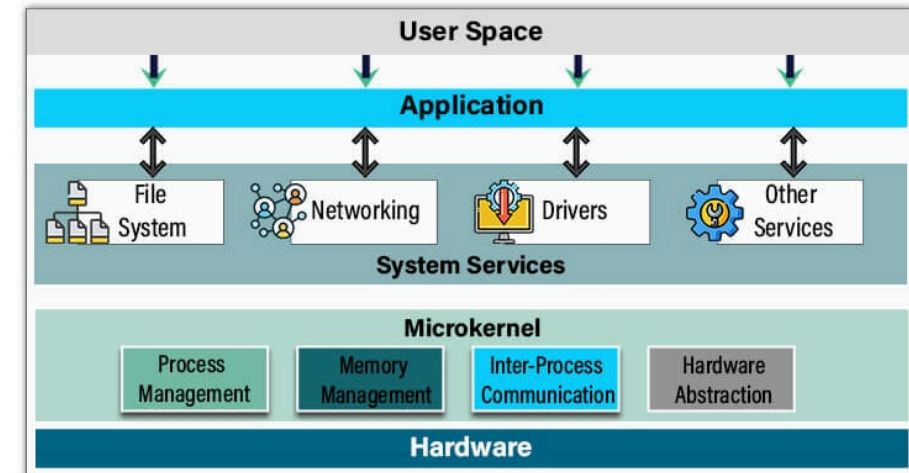
- Keamanan **lebih tinggi** dengan isolasi layanan
- Pembaruan sistem **lebih mudah** tanpa restart
- Portabilitas **lebih baik** ke berbagai platform

## Kekurangan

- Performa **lebih lambat** karena komunikasi antar proses
- Penggunaan memori **lebih besar**
- Implementasi **lebih kompleks**

## Microkernel in Operating System

A Minimalistic kernel with essential functions



EDUCBA



**Contoh Sistem Operasi**  
Minix, QNX, Mach, L4



**Mikrokernel** Fungsi dasar: IPC, penjadwalan dasar



**Server File** Mengelola sistem file



**Server Driver** Mengontrol perangkat keras



**Server Jaringan** Menangani komunikasi jaringan

# Arsitektur Client-Server dan Virtual Mesin

## Client-Server

Sistem yang memisahkan fungsi menjadi **klien** (meminta layanan) dan **server** (menyediakan layanan)



### Kelebihan

Dapat digunakan pada **sistem terdistribusi**, fleksibel, mudah dikembangkan



### Kekurangan

Performa **lebih lambat**, komunikasi bisa menjadi bottleneck



### Contoh

Windows 2000 Server, Linux

## Virtual Mesin

Sistem yang melakukan **simulasi mesin nyata** untuk menjalankan sistem operasi berbeda pada satu komputer



### Kelebihan

Pengembangan sistem **aman**, fleksibilitas tinggi, isolasi sistem



### Kekurangan

Implementasi **sulit**, performa lebih rendah, overhead tinggi



### Contoh

Windows NT, VMware, VirtualBox



# Arsitektur Berorientasi Objek

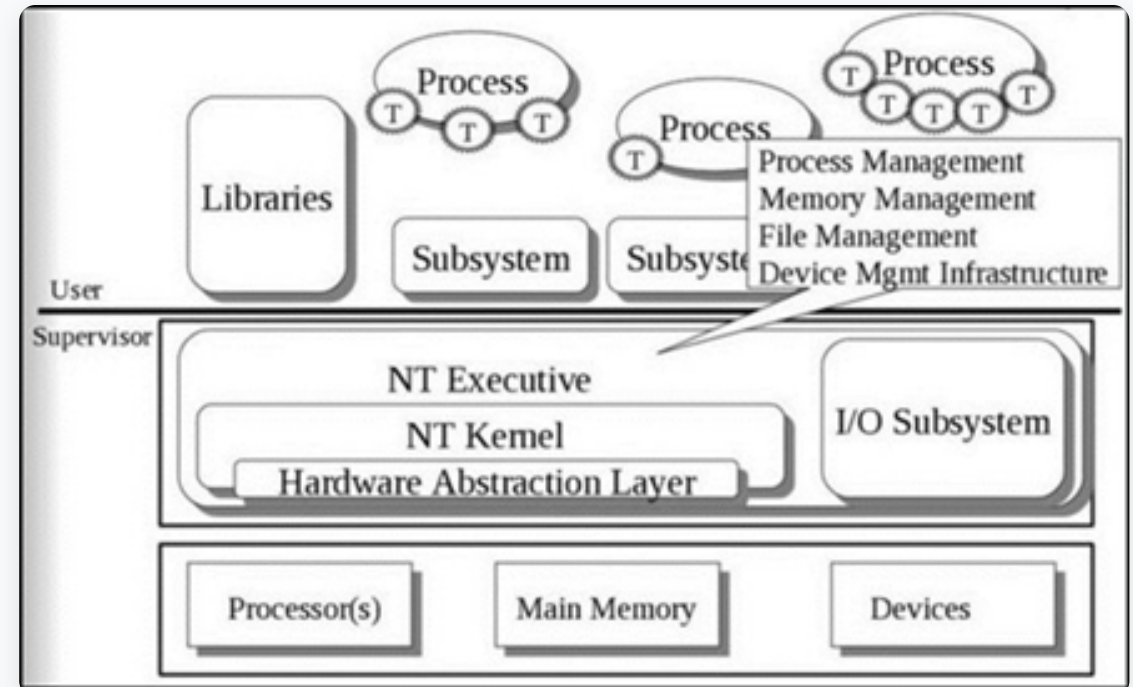
Sistem operasi yang merealisasikan layanan sebagai **kumpulan objek** yang mengkapsulasi data dan operasi

## Kelebihan

- Struktur **terorganisir** dengan baik
- Keamanan **lebih baik** dengan enkapsulasi
- Kemudahan **pengembangan** dan pemeliharaan

## Kekurangan

- Performa **lebih lambat** karena overhead objek
- Penggunaan memori **lebih besar**
- Implementasi **lebih kompleks**



## Contoh Sistem Operasi

Windows NT, Amoeba, Eden, X-Kernel



**Objek File** Mengelola operasi file



**Objek Memori** Mengelola alokasi memori



**Objek Proses** Mengatur eksekusi proses

## Kesimpulan

Struktur sistem operasi adalah **fondasi** yang menentukan cara kerja dan efisiensi sistem komputer



### Layanan Sistem Operasi

Eksekusi program, operasi I/O, manipulasi file, komunikasi, dan deteksi kesalahan



### Antarmuka Sistem

User interface, system calls, dan system services sebagai jembatan antara pengguna dan kernel



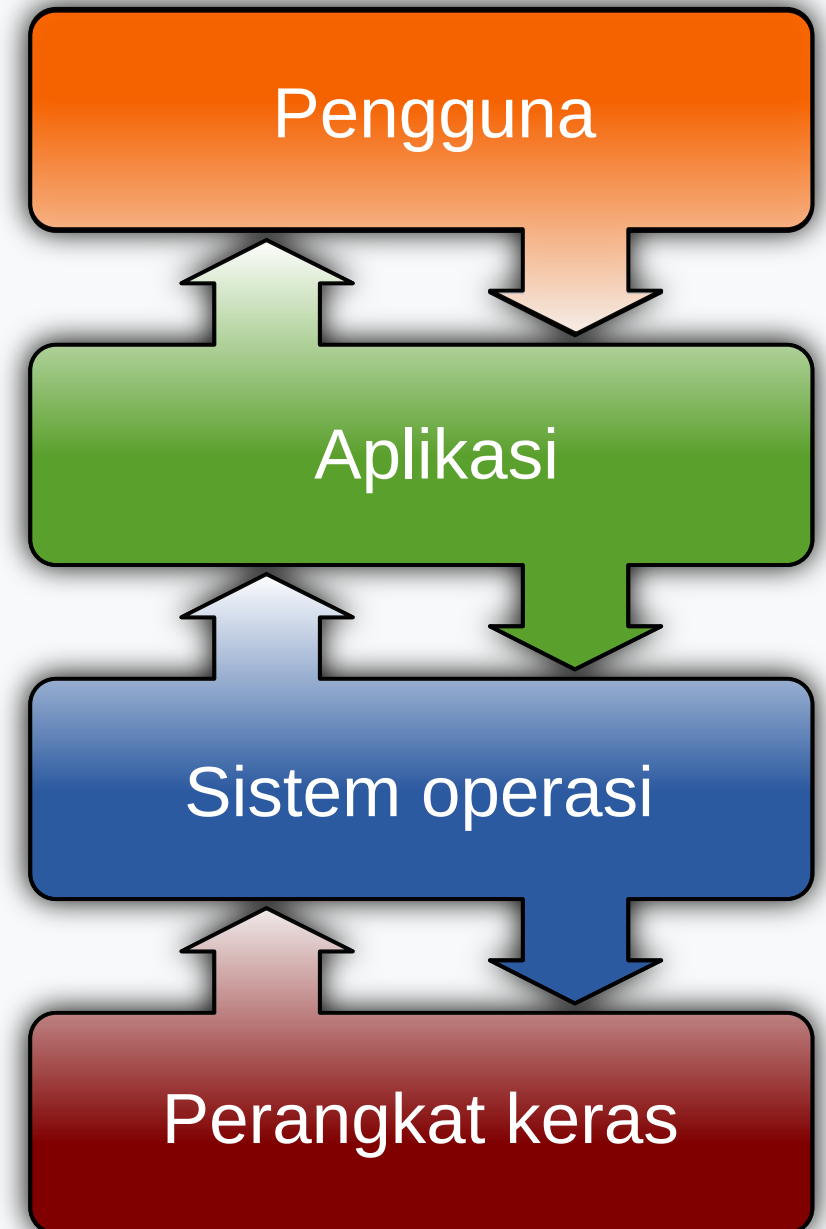
### Arsitektur Sistem Operasi

Monolitik, berlapis, mikrokernel, client-server, virtual mesin, dan berorientasi objek



### Implementasi dan Debugging

Proses desain, pembangunan, booting, dan debugging untuk sistem operasi yang andal



### Pemahaman Struktur OS

Memahami struktur sistem operasi membantu dalam **pengembangan aplikasi** yang lebih efisien dan **troubleshooting** sistem